RAMCO INSTITUTE OF TECHNOLOGY
RAJAPALAYAM
Dept. of Artificial Intelligence and Data Science

A Consensus Based AI-Powered Smart Contract Security Auditor

Presented By

Siddharth N.R & 953622243093

Project Supervisor
Dr Mariappan E
Associate Professor III
AD

Abstract

- **Sentient Audit** is an AI-powered smart contract security auditing platform that combines Machine Learning (ML) classification, Generative AI (**Google Gemini 2.5 Flash**), and deterministic rule-based analysis to detect vulnerabilities in Solidity smart contracts.
- The system implements a **4-Stage Hybrid Scanning Pipeline**:

- **ML Scan** — TF-IDF + Random Forest multi-label classification
- **AI Semantic Analysis**
- **AI Logic Flow & Reentrancy Analysis**
- **AI Edge Case & Input Validation Analysis**

System Architecture

Layer	Technology	Purpose
Frontend	React 18 + TypeScript + Vite	Code input, real-time progress tracking, results visualization
UI Components	shadcn/ui + Tailwind CSS + Recharts	Security score charts, severity badges, code highlighting
ML Backend	Python + Flask + scikit-learn	TF-IDF + Random Forest classifier + rule detectors
AI Engine	Google Gemini 2.5 Flash API	3 specialized AI analysis passes
Code Regenerator	Gemini AI	Automatic secure code rewriting
API Communication	REST (POST /api/scan)	Frontend ↔ Backend communication

Detection Pipeline

Stage 1 — ML Classification
Pattern recognition using TF-IDF + Random Forest

Stage 2 — AI Semantic Analysis
Function visibility, modifiers, access control issues

Stage 3 — AI Logic Flow & Reentrancy Analysis
State update order, external calls, reentrancy risks

Stage 4 — AI Edge Case & Input Validation
Zero values, overflow conditions, boundary cases

All findings are merged using consensus and confidence boosting.

ML Classification Workflow

Input: Raw Solidity Source Code
↓
Tokenization
- Remove comments
- Normalize tokens
- Classify keywords, EXTCALL_*, GUARD_*
- Build compound tokens (msg.sender, tx.origin)

↓
Feature Extraction
- TF-IDF (max_features=2000, n-gram=1–3, sublinear TF)
- Structural features (function count, pragma, visibility)

↓
Classification
- OneVsRest(RandomForest)
- Multi-label threshold > 0.3

↓
Rule-Based Detection (17 rules)
↓
Merge ML + Rules
↓
Output: Structured Vulnerability Report

Mathematical Model

Security Score

$$\text{Score} = \max(0, \min(100, 100 - \Sigma(W_{?})))$$

TF-IDF Formula

$$\text{TF-IDF}(t, d) = \text{TF}(t, d) \times \text{IDF}(t)$$
$$\text{TF}(t, d) = 1 + \log(f(t, d))$$
$$\text{IDF}(t) = \log(N / df(t)) + 1$$

Multi-Label Classification

For each label L_i:
Train binary classifier $P(L_i \mid X)$
If $P(L_i \mid X) > 0.3$

Applied Engineering Principles

- Defense in Depth through 4-layer detection pipeline prevents single-point failure.

- Separation of Concerns ensures frontend, ML backend, and AI engine are independently scalable.

- Graceful Degradation enables fallback scanning if backend services fail.

- Statistical robustness achieved using median scoring and consensus-based filtering.

Identification of Alternate Solutions

- Static analysis tools (Mythril/Slither) require complex setup and lack AI semantic reasoning.

- Pure Deep Learning requires massive datasets and GPU resources for effective training.

- Pure LLM-based systems suffer from hallucination and inconsistent detection.

- Hybrid ML + Rules + AI approach chosen for best balance of speed, accuracy, and reliability.

Problem-Solving Approach

Problem Identification — Smart contract vulnerabilities cause billions in losses annually (DAO hack: $60M, Parity wallet: $280M). Manual auditing is slow, expensive, and human-error-prone.

Requirement Analysis— Need fast, accessible, comprehensive detection covering 20+ vulnerability classes with actionable fix recommendations.

Decomposition — Broke the problem into 4 independent detection layers: - Layer 1: ML classification (pattern recognition from training data) - Layer 2: AI semantic analysis (function visibility, access control) - Layer 3: AI logic flow analysis (reentrancy, state ordering) - Layer 4: AI edge case analysis (zero values, overflow, gas limits)

Ensemble Strategy — Union-merge all findings with confidence boosting for cross-validated results, eliminating false negatives.

Consensus Mechanism — Triple-run each AI pass to eliminate hallucinations, keeping only the most detailed version of each finding.

Automated Remediation — Beyond detection, the system generates fixed code using AI, reducing developer burden.

Progress of Work

Completed Features:

- Built Solidity tokenizer, curated dataset, and trained multi-label ML classifier.
- Developed 17 deterministic rule detectors for guaranteed vulnerability coverage.
- Integrated Gemini AI with triple-run consensus engine.
- Developed interactive dashboard with real-time audit results and secure code regeneration.

Upcoming Features:

- **NFT Support** – Mints ERC-721 NFT If Security Score Is Above 85%

Technologies Used & Technical Issues Addressed

Technologies Used:

- React, TypeScript, Tailwind, and Recharts for modern responsive UI.
- Python, Flask, and scikit-learn for ML-based backend processing.
- Google Gemini 2.5 Flash for advanced AI-driven vulnerability analysis.
- Implemented rate-limit handling, caching, and CORS configuration for reliability.

Technical Challenges:

AI Hallucination — Solved via triple-run consensus; findings not confirmed across runs are ranked lower

API Rate Limiting (429)* — Exponential backoff up to 90 seconds with Retry-After header support

Cold Start Latency — ML model trains at server startup (~2 seconds) and caches in memory as singleton

Cross-Origin Request — Flask-CORS configured for localhost development origins

Graceful Degradation — Client-side fallback scanner if Python backend is unavailable

Commitment to Professional Ethics

- **Transparency** - Each finding shows detection method (`ml`, `rules`, or `ml+rules`), confidence score, CWE-ID references, and confirmedBy count — users understand how and why each vulnerability was detected
- **Accessibility -** Web-based interface makes security auditing accessible to all developers, not just security experts; removes the cost barrier of professional audits
- **Data Privacy -** Code is processed locally on the ML backend or sent to Gemini API transiently; no user code is stored permanently
- **No False Confidence -** System clearly reports confidence levels (high/medium/low), triple-run confirmation counts, and security scores — does not overstate its analysis reliability
- **Open Standards -** Uses industry-standard CWE (Common Weakness Enumeration) and SWC (Smart Contract Weakness Classification) identifiers for all vulnerability classifications
- **Beneficence -** Aims to protect end-users of deployed smart contracts by helping developers identify and fix security flaws before deployment, preventing financial losses

SCREENSHOTS OF CODE AND OUTPUT

Machine Learning Model

sentient-audit > ml-backend > model.py

```python
"""
ML Vulnerability Detection Model
TF-IDF + RandomForest multi-label classifier for Solidity vulnerability detection.
Trains at server startup (~2 seconds) on curated samples and caches in memory.
"""

import re
import numpy as np
from typing import List, Dict, Tuple, Set
from sklearn.feature_extraction.text import TfidfVectorizer
from sklearn.ensemble import RandomForestClassifier
from sklearn.multiclass import OneVsRestClassifier
from sklearn.preprocessing import MultiLabelBinarizer

from training_data import TRAINING_SAMPLES, ALL_LABELS, VULNERABILITY_INFO, SAFE
from solidity_tokenizer import tokenize_solidity, extract_functions, extract_contract_features
from vulnerability_rules import run_all_rules


class SolidityVulnerabilityModel:
    """
    Multi-label vulnerability classifier using TF-IDF + Random Forest.
    Combines ML predictions with deterministic rule-based detection for 99% coverage.
    """

```

SCREENSHOTS OF CODE AND OUTPUT

Trained Data

sentient-audit > ml-backend > training_data.py

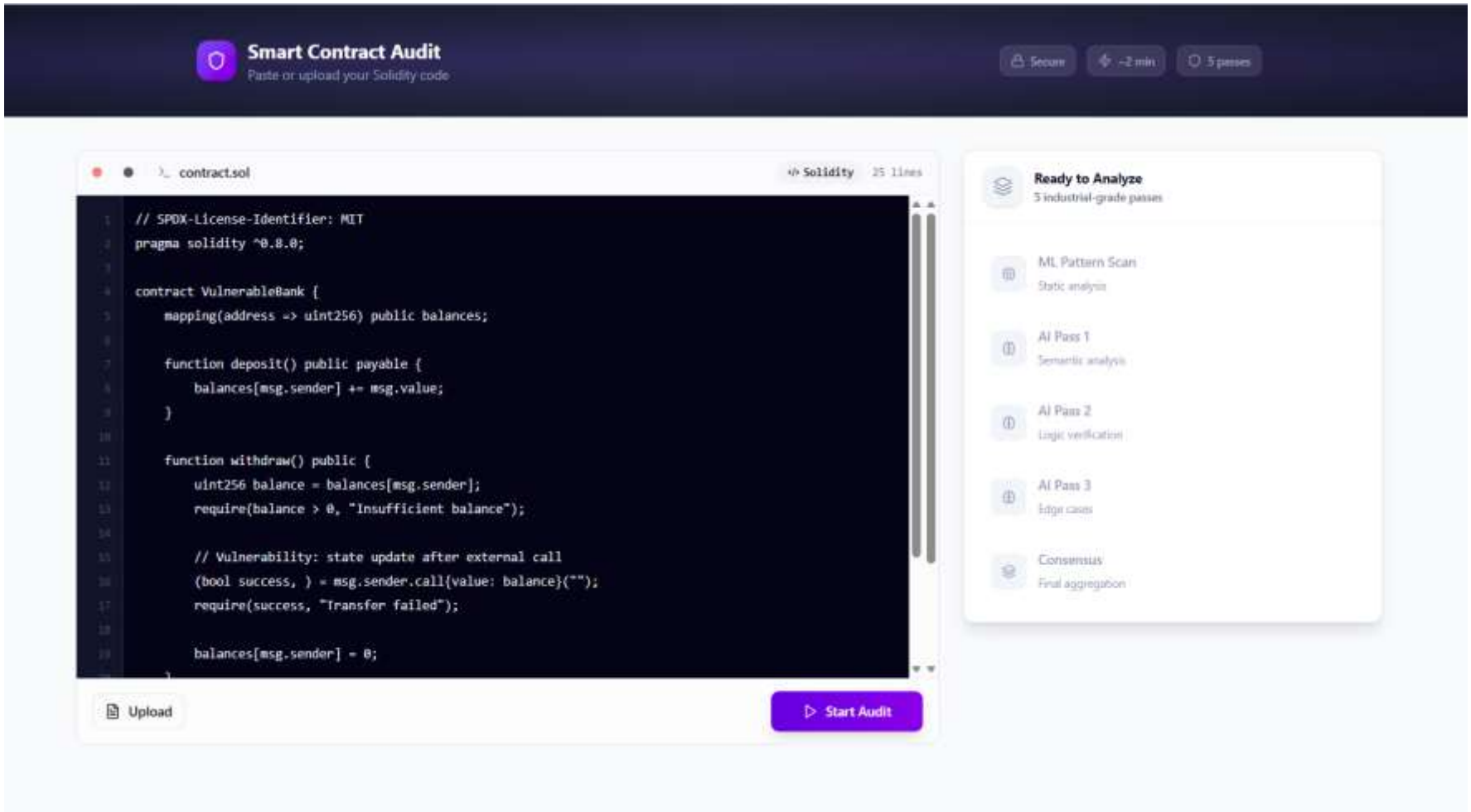
```python
"""
Curated training dataset of Solidity vulnerability samples.
Each sample maps a code snippet to its vulnerability label(s).
Used by the ML model for TF-IDF + RandomForest training.
"""

# Label constants
REENTRANCY = "reentrancy"
ACCESS_CONTROL = "missing_access_control"
UNCHECKED_CALL = "unchecked_call_return"
TX_ORIGIN = "tx_origin_auth"
DELEGATECALL = "dangerous_delegatecall"
INTEGER_OVERFLOW = "integer_overflow"
ZERO_ADDRESS = "missing_zero_check"
SELFDESTRUCT = "unprotected_selfdestruct"
TIMESTAMP = "timestamp_dependence"
FLOATING_PRAGMA = "floating_pragma"
UNBOUNDED_LOOP = "unbounded_loop"
FRONT_RUNNING = "front_running"
DOS_REVERT = "dos_with_revert"
FORCE_ETHER = "force_ether_receive"
SIGNATURE_REPLAY = "signature_replay"
FLASH_LOAN = "flash_loan_attack"
STORAGE_COLLISION = "storage_collision"
UNINITIALIZED_STORAGE = "uninitialized_storage"
PRIVATE_DATA = "private_data_exposure"
GAS_GRIEFING = "gas_griefing"
SAFE = "safe"

ALL_LABELS = [
    REENTRANCY, ACCESS_CONTROL, UNCHECKED_CALL, TX_ORIGIN, DELEGATECALL,
    INTEGER_OVERFLOW, ZERO_ADDRESS, SELFDESTRUCT, TIMESTAMP, FLOATING_PRAGMA,
    UNBOUNDED_LOOP, FRONT_RUNNING, DOS_REVERT, FORCE_ETHER, SIGNATURE_REPLAY,
    FLASH_LOAN, STORAGE_COLLISION, UNINITIALIZED_STORAGE, PRIVATE_DATA, GAS_GRIEFING, SAFE
```

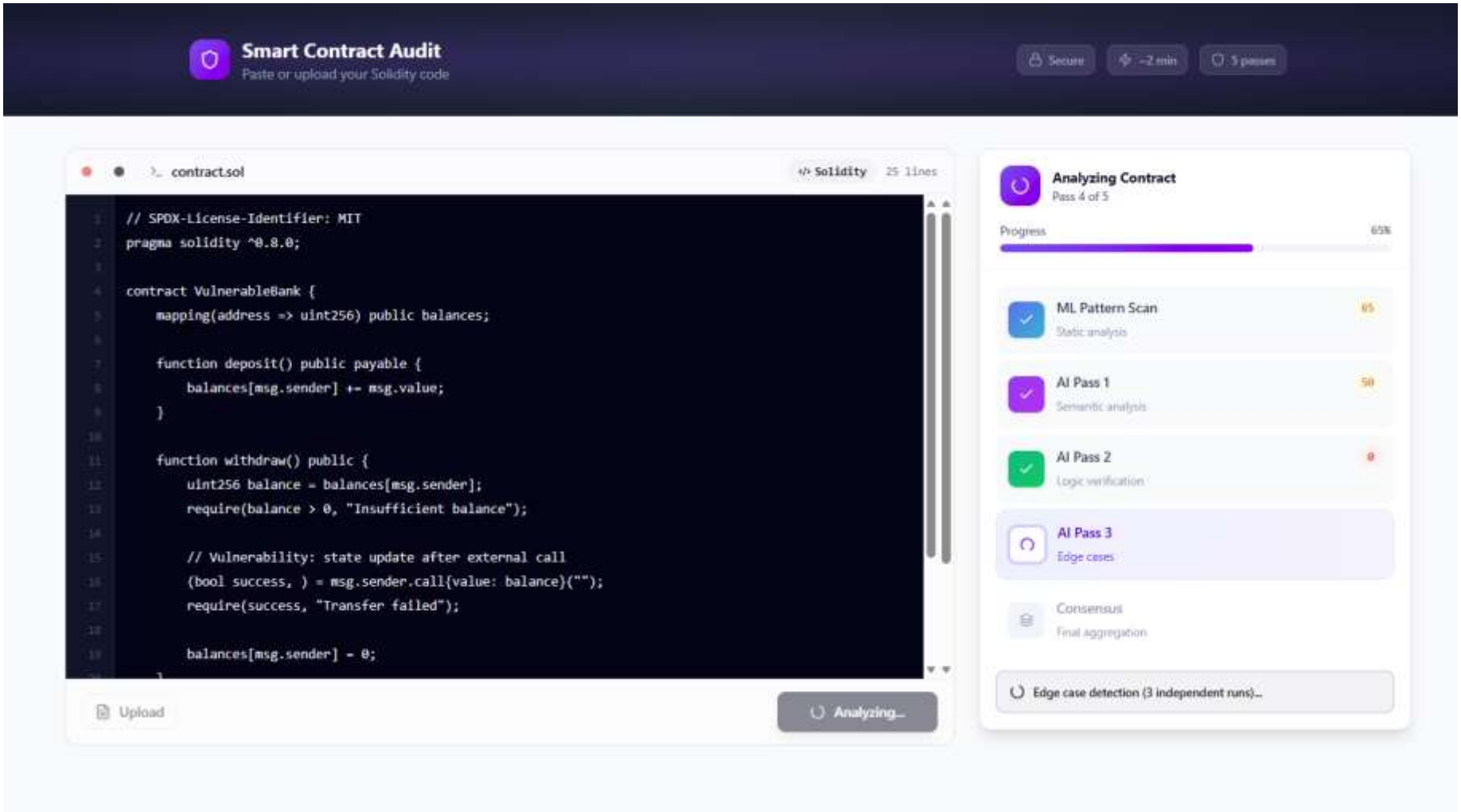
Project Screenshots

Solidity Code Upload Page



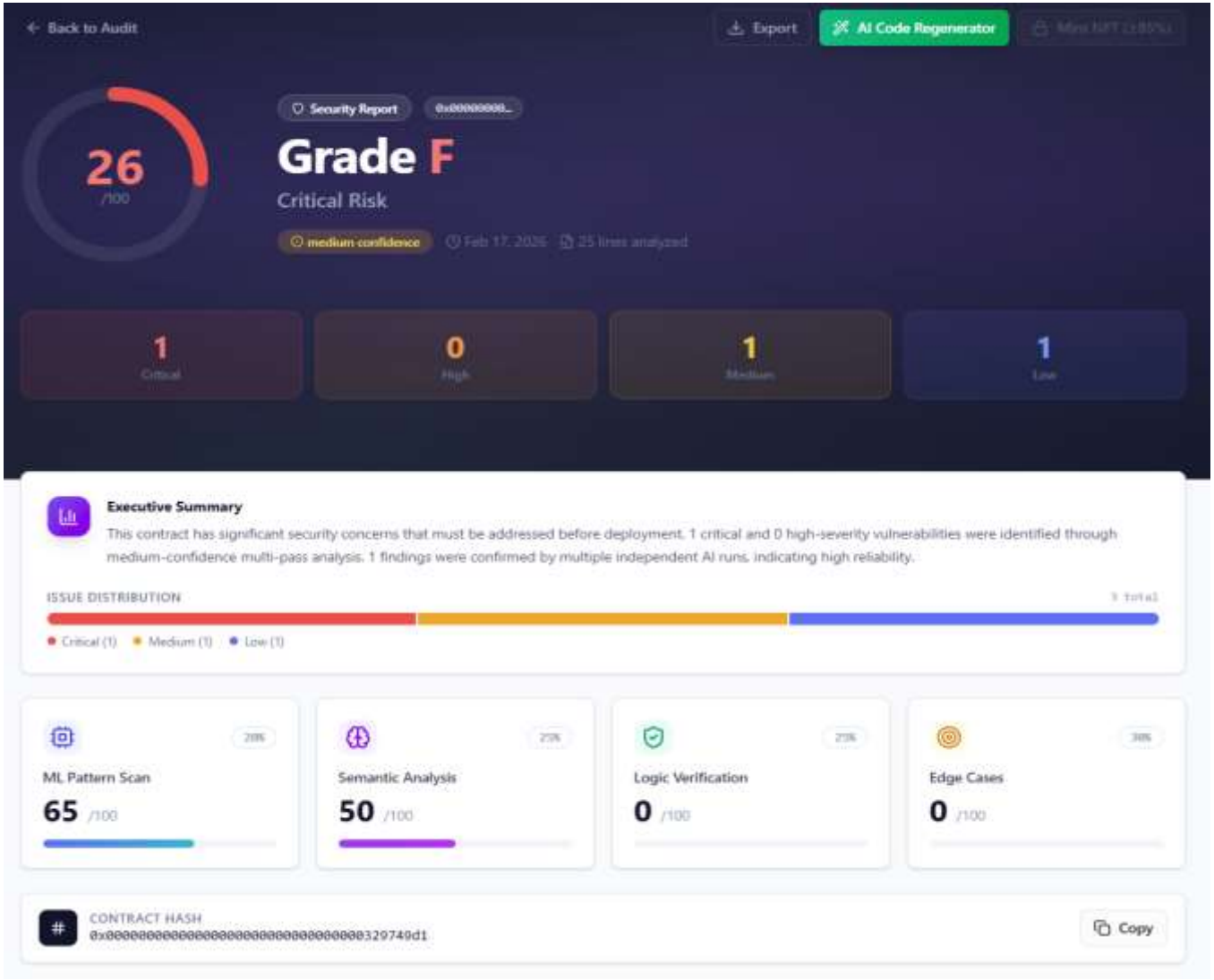
SCREENSHOTS OF CODE AND OUTPUT

Running Auditing Of Smart Contract Code



SCREENSHOTS OF CODE AND OUTPUT

Report Page



SCREENSHOTS OF CODE AND OUTPUT

Additional Detailed Report

Detailed Findings

Sorted by severity • Click to expand

3 issues

01 Critical | Access Control | L16 | CWE-841

Reentrancy Vulnerability

The `withdraw()` function is vulnerable to a reentrancy attack. The contract sends Ether to `msg.sender` via an external call (`msg.sender.call{value: balance}("")`) before updating the sender's balance to zero (`balances[msg.sender] = 0;`). If `msg.sender` is a malicious contract, it can re-enter the `withdraw()`...

Details | Impact | Fix | Code

Description

The `withdraw()` function is vulnerable to a reentrancy attack. The contract sends Ether to `msg.sender` via an external call (`msg.sender.call{value: balance}("")`) before updating the sender's balance to zero (`balances[msg.sender] = 0;`). If `msg.sender` is a malicious contract, it can re-enter the `withdraw()` function multiple times before the balance is reset, allowing it to drain more Ether than it legitimately holds.

02 Medium | Event Patterns | L13 | CWE-1006 | 2/3 runs

Missing Event for Withdrawal

The `withdraw()` function performs a transfer of Ether and resets the user's balance, but it does not emit an event to log this critical state change. Emitting events is essential for off-chain monitoring, indexing, and providing transparency about transactions.

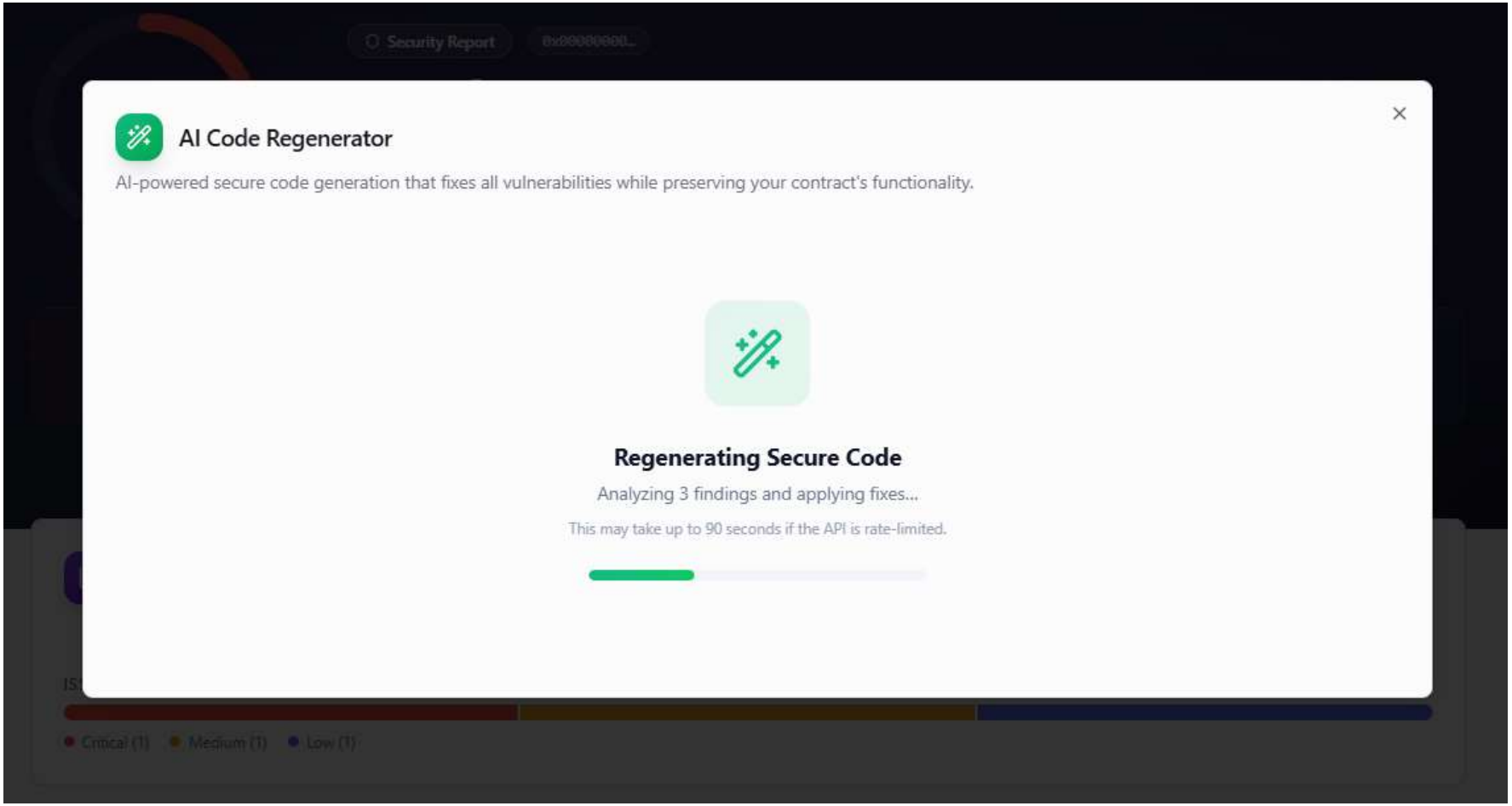
03 Low | Best Practices | L2 | CWE-664

Floating Pragma

Floating pragma allows compilation with different compiler versions.

SCREENSHOTS OF CODE AND OUTPUT

Code Regenerator



Project Completion Plan – NFT Integration

- **Remaining Work:** Develop ERC-721 NFT smart contract with minting, royalties (EIP-2981), IPFS metadata integration, and security validation using Sentient Audit.

- **Integration & Testing:** Connect NFT module with AI audit engine, deploy on testnet, and perform vulnerability, gas optimization, and edge-case testing.

- **Timeline:** 4-Week Plan – (W1) Contract Development, (W2) Audit Integration & Testnet Deployment, (W3) UI & Optimization, (W4) Final Testing & Documentation.

- **Expected Deliverables:** Fully secure NFT-enabled smart contract system, AI-generated audit reports, integrated web dashboard, and complete technical documentation with demo.

Thank You